Université Sorbonne Paris Nord

SAE 2-01 Conception et implémentation d'une base de données : Notations

BUT1 SD
Groupe Mirzakhani

Réalisé Par :
Mese Sidar
Diop Abou

Année Universitaire 2022 – 2023

Sommaire

1. *Modélisation et script de création « sans AGL »* 3
2. *Modélisation et script de création « avec AGL »* 5
3. *Peuplement des tables de la base de données* 11
4. *Requête particulièrement intéressante* 15

1. Modélisation et script de création « sans AGL »

Pour la réalisation de cette SAE, nous avons utilisé le logiciel de diagramme draw.io (diagrams.net) et MySQL Workbench comme AGL.

Modèle entité-associations

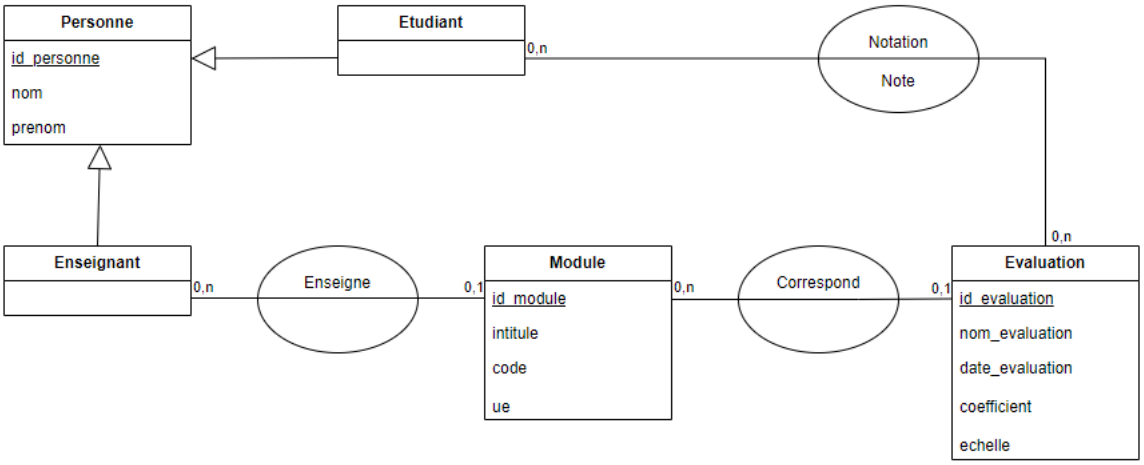


Schéma relationnel

Personne (id_personne, nom, prenom)
Enseignant (id_enseignant) où id_enseignant fait référence à Personne
Etudiant (id_etudiant) où id_etudiant fait référence à Personne
Module (id_module, intitule, code, ue, id_enseignant) où id_enseignant fait référence à Enseignant
Evaluation (id_evaluation, nom_evaluation, date_evaluation, coefficient, echelle, id_module) où idmodule fait référence à Module
Notation (id_etudiant, id_evaluation, note)
Où id_etudiant fait référence à Etudiant
Où id_evaluation fait référence à Evaluation

Script de création des tables

```
CREATE TABLE Personne (
  id_personne INTEGER PRIMARY KEY,
  nom VARCHAR,
  prenom VARCHAR
) ;

CREATE TABLE Enseignant (
  id_enseignant INTEGER PRIMARY KEY,
  FOREIGN KEY (id_enseignant) REFERENCES Personne (id_personne)
) ;

CREATE TABLE Etudiant (
  id_etudiant INTEGER PRIMARY KEY,
  FOREIGN KEY (id_etudiant) REFERENCES Personne (id_personne)
) ;

CREATE TABLE Module (
  id_module INTEGER PRIMARY KEY,
  intitule_module VARCHAR,
```

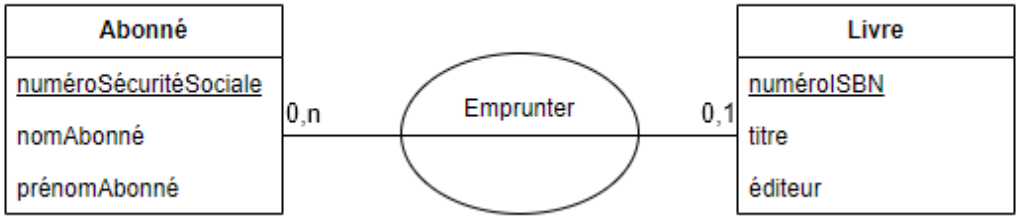
```sql
  code VARCHAR,
  ue VARCHAR,
  id_enseignant INTEGER,
  FOREIGN KEY (id_enseignant) REFERENCES Enseignant (id_enseignant)
) ;

CREATE TABLE Evaluation (
  id_evaluation SERIAL PRIMARY KEY,
  nom_evaluation VARCHAR,
  date_evaluation DATE,
  coefficient FLOAT,
  echelle INTEGER,
  id_module INTEGER,
  FOREIGN KEY (id_module) REFERENCES Module (id_module)
) ;

CREATE TABLE Notation (
  id_etudiant INTEGER,
  id_evaluation INTEGER,
  note FLOAT,
  FOREIGN KEY (id_etudiant) REFERENCES Etudiant (id_etudiant),
  FOREIGN KEY (id_evaluation) REFERENCES Evaluation (id_evaluation),
  PRIMARY KEY (id_etudiant, id_evaluation)
) ;
```

Dans cette section, Sidar a réalisé le modèle entité-association, tandis qu'Abou s'est occupé de la conception du schéma relationnel. Le script de création des tables, quant à lui, a été réalisé ensemble.

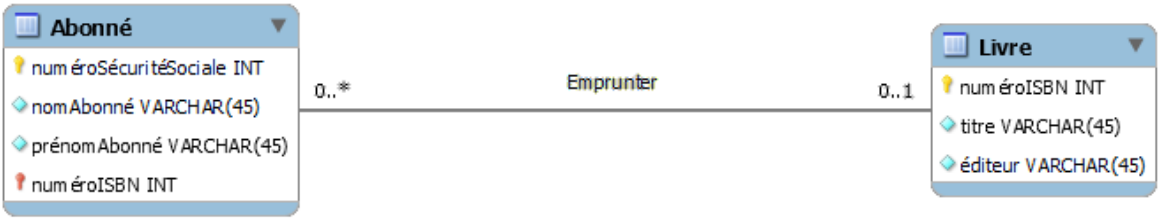
2. Modélisation et script de création « avec AGL »

Association fonctionnelle

Avec Draw.io

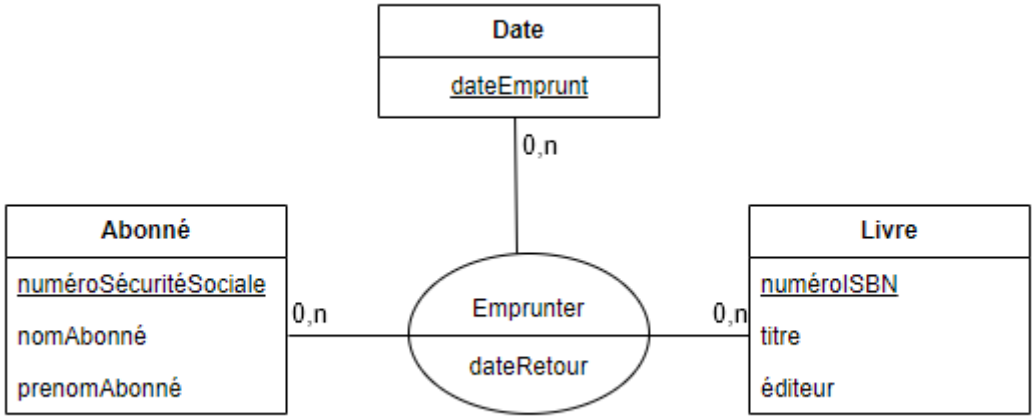


Avec MySQL Workbench

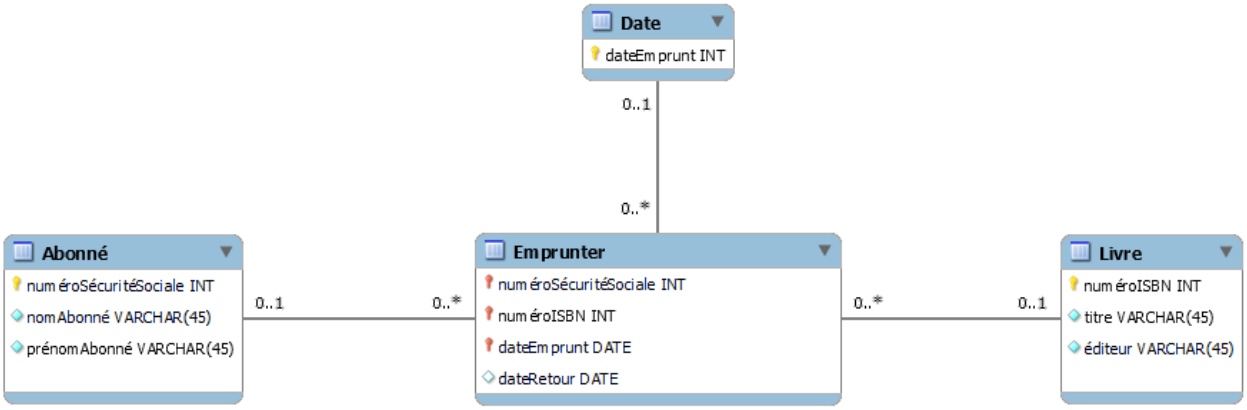


Association maillée

Avec Draw.io



Avec MySQL Workbench

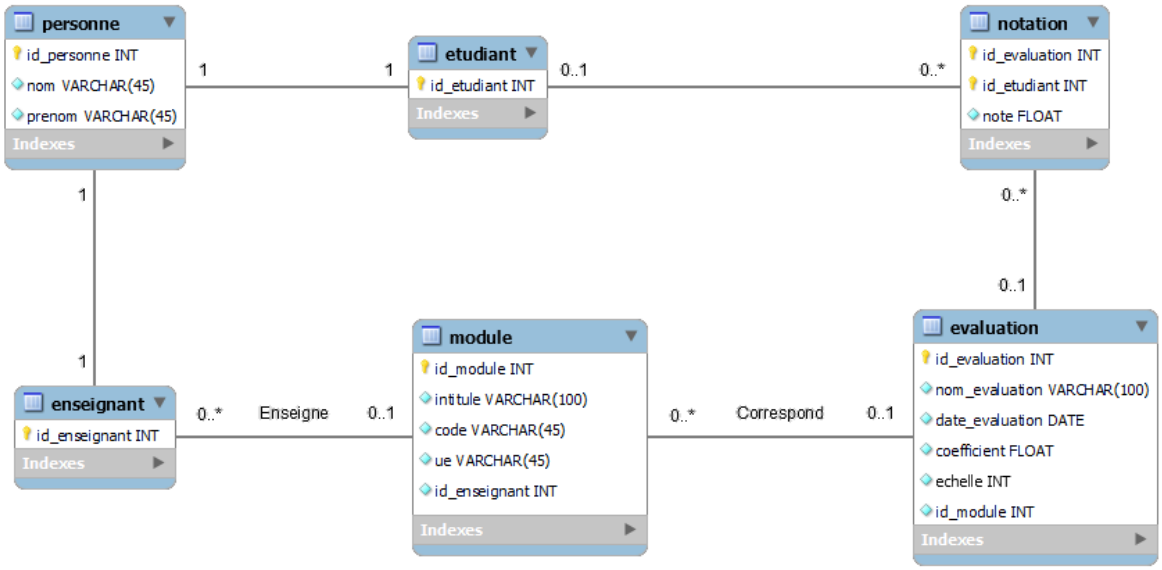


Commentaires

En comparant le modèle Entité-Association réalisé avec Draw.io et MySQL Workbench, on remarque que dans MySQL Workbench, lors de la création des tables, il est nécessaire de renseigner le type de données des attributs, et ces types de données s'affichent dans les tables une fois créées. En revanche, dans Draw.io, qui est un logiciel de diagramme plus généraliste, il n'existe pas de fonctionnalité spécifique pour afficher les types de données dans les tables. Toutefois, on peut utiliser du texte ou des annotations à côté des attributs pour indiquer les types de données si nécessaire. En ce qui concerne les relations entre les entités, Draw.io offre la possibilité d'utiliser des flèches et des lignes pour indiquer les relations. Cependant, il n'y a pas de notation standardisée spécifique pour les relations dans Draw.io. Il est donc nécessaire d'utiliser des formes et des annotations pour représenter la nature de la relation, telle que la cardinalité (1:1, 1:N, etc.).En revanche, MySQL Workbench propose plusieurs notations standardisées pour représenter les relations dans le modèle Entité-Association. Parmi ces notations, on trouve la notation "Crow's Foot" (pied de corbeau), la notation classique et la notation UML. Ces notations fournissent des symboles spécifiques tels que des losanges pour représenter les relations, et elles incluent des indicateurs de cardinalité pour spécifier les types de relations. Dans draw.io, il est possible de créer une représentation visuelle d'un schéma de base de données, mais la définition des contraintes de nullité pour les attributs des tables n'est pas prise en charge directement. En revanche, avec l'utilisation de MySQL Workbench, il est possible de spécifier les attributs et indiquer explicitement s'ils peuvent contenir des valeurs "NULL" ou s'ils doivent être "non NULL". En termes de facilité d'utilisation, Draw.io permet une grande flexibilité dans la conception et la personnalisation des diagrammes, mais il nécessite souvent une configuration manuelle et des annotations pour représenter les types de données et les relations de manière précise. D'un autre côté, MySQL Workbench est plus axé sur la modélisation de bases de données et offre une

expérience plus automatisée. Il dispose d'outils spécifiques pour définir les types de données, les relations et les cardinalités, ce qui facilite la création des tables et la génération automatique des scripts SQL correspondants.

Modèle entité-associations réalisé avec l'AGL



Script SQL de création des tables généré automatiquement par l'AGL

```
SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS,
FOREIGN_KEY_CHECKS=0;
SET@OLD_SQL_MODE=@@SQL_MODE,SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRAN
S_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_
ENGINE_SUBSTITUTION';

-- -----------------------------------------------------
-- Table `personne`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `personne` (
  `id_personne` INT NOT NULL,
  `nom` VARCHAR(45) NOT NULL,
  `prenom` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`id_personne`))
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;

-- -----------------------------------------------------
-- Table `enseignant`
```

```sql
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `enseignant` (
  `id_enseignant` INT NOT NULL,
  PRIMARY KEY (`id_enseignant`),
  CONSTRAINT `id_enseignant`
    FOREIGN KEY ()
    REFERENCES `personne` ()
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;

-- -----------------------------------------------------
-- Table `etudiant`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `etudiant` (
  `id_etudiant` INT NOT NULL,
  PRIMARY KEY (`id_etudiant`),
  CONSTRAINT `id_etudiant`
    FOREIGN KEY ()
    REFERENCES `personne` ()
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;

-- -----------------------------------------------------
-- Table `module`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `module` (
  `id_module` INT NOT NULL,
  `intitule` VARCHAR(100) NOT NULL,
  `code` VARCHAR(45) NOT NULL,
  `ue` VARCHAR(45) NOT NULL,
  `id_enseignant` INT NOT NULL,
  PRIMARY KEY (`id_module`),
  CONSTRAINT `id_enseignant`
    FOREIGN KEY ()
    REFERENCES `enseignant` ()
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;

-- -----------------------------------------------------
-- Table `evaluation`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `evaluation` (
```

```sql
  `id_evaluation` INT NOT NULL,
  `nom_evaluation` VARCHAR(100) NOT NULL,
  `date_evaluation` DATE NOT NULL,
  `coefficient` FLOAT NOT NULL,
  `echelle` INT NOT NULL,
  `id_module` INT NOT NULL,
  PRIMARY KEY (`id_evaluation`),
  CONSTRAINT `id_module`
    FOREIGN KEY ()
    REFERENCES `module` ()
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;

-- -----------------------------------------------------
-- Table `notation`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `notation` (
  `id_evaluation` INT NOT NULL,
  `id_etudiant` INT NOT NULL,
  `note` FLOAT NOT NULL,
  PRIMARY KEY (`id_evaluation`, `id_etudiant`),
  CONSTRAINT `id_etudiant`
    FOREIGN KEY ()
    REFERENCES `etudiant` ()
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `id_evaluation`
    FOREIGN KEY ()
    REFERENCES `evaluation` ()
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB
DEFAULT CHARACTER SET = utf8mb4
COLLATE = utf8mb4_0900_ai_ci;

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
```

Différence entre les scripts produits manuellement et automatiquement

Les différences entre le script manuel et le script automatique en SQL se situent au niveau de la saisie des types de données. Dans le script manuel, l'utilisateur est contraint de spécifier les types de données tels que "INTEGER", "VARCHAR", "DATE" ou "FLOAT" et la gestion des valeurs "NULL" ET "NOT NULL" nécessitant une

connaissance préalable et une saisie manuelle. En revanche, le script automatique propose directement des types de données spécifiques en fonction des attributs, facilitant la sélection en évitant la saisie manuelle. Par exemple, il peut suggérer automatiquement "VARCHAR(45)" pour un attribut de texte. Le script automatique simplifie le processus en déduisant les valeurs "NULL" et "NOT NULL" en fonction des attributs. En ce qui concerne les clés étrangères, le script manuel définit les contraintes de clé étrangère lors de la création des tables en utilisant la clause "FOREIGN KEY", tandis que le script automatique les définit après la création des tables en utilisant la clause "CONSTRAINT" avec des références vides, qui sont ajoutées ultérieurement. Les options de contrainte diffèrent également, le script manuel ne spécifie pas explicitement les actions à prendre en cas de suppression ou de mise à jour des enregistrements liés par des clés étrangères, tandis que le script automatique utilise les options "ON DELETE NO ACTION" et "ON UPDATE NO ACTION" pour indiquer qu'aucune action ne doit être effectuée. Enfin, quant aux options de moteur de stockage, le script manuel ne spécifie pas le moteur de stockage à utiliser pour les tables, tandis que le script automatique utilise le moteur de stockage InnoDB, et il définit également un jeu de caractères par défaut ("utf8mb4") et une intercalation spécifique ("utf8mb4_0900_ai_ci") pour les tables.

Dans cette section, les associations fonctionnelles, maillées et les comparaisons a été réalisé par Abou. En revanche, le modèle entité-association, le script de création de tables, ainsi que l'analyse des différences entre les scripts produits manuellement et automatiquement a été réalisé par Sidar.

3. Peuplement des tables de la base de données

Etape 1 : Pour commencer, nous avons mis en place une table appelée "grandetable" qui a été conçue pour stocker l'ensemble des attributs.

```
CREATE TABLE grandetable (
  id_enseignant INTEGER,
  nom_enseignant VARCHAR,
  prenom_enseignant VARCHAR,
  id_module INTEGER,
  code VARCHAR,
  ue VARCHAR,
  intitule_module VARCHAR,
  nom_evaluation VARCHAR,
  date_evaluation DATE,
  coefficient FLOAT,
  echelle INTEGER,
  note FLOAT,
  id_etudiant INTEGER,
  nom_etudiant VARCHAR,
  prenom_etudiant VARCHAR ) ;
```

```
postgres=# \d grandetable;
                        Table % public.grandetable ╗
      Colonne       |        Type         | Collationnement | NULL-able | Par dÚfaut
--------------------+---------------------+-----------------+-----------+-----------
 id_enseignant      | integer             |                 |           |
 nom_enseignant     | character varying   |                 |           |
 prenom_enseignant  | character varying   |                 |           |
 id_module          | integer             |                 |           |
 code               | character varying   |                 |           |
 ue                 | character varying   |                 |           |
 intitule_module    | character varying   |                 |           |
 nom_evaluation     | character varying   |                 |           |
 date_evaluation    | date                |                 |           |
 coefficient        | double precision    |                 |           |
 echelle            | integer             |                 |           |
 note               | double precision    |                 |           |
 id_etudiant        | integer             |                 |           |
 nom_etudiant       | character varying   |                 |           |
 prenom_etudiant    | character varying   |                 |           |
```

Etape 2 : Par la suite, nous avons utilisé la fonction "COPY" pour insérer les données provenant du fichier "data.csv" directement dans la table "grandetable".

```
\copy grandetable
FROM 'C:\Program Files\PostgreSQL\14\scripts\data.csv' WITH
DELIMITER ';' CSV HEADER;
```

```
postgres=#
COPY 6087
```

Etape 3 : Ensuite, nous avons procédé à la création des différentes tables nécessaires pour organiser les données de manière structurée. Ces tables comprennent "Personne", "Enseignant", "Etudiant", "Module", "Evaluation" et "Notation".

```sql
-- Création de la table "Personne"
CREATE TABLE Personne (
  id_personne INTEGER PRIMARY KEY,
  nom VARCHAR,
  prenom VARCHAR
);

-- Création de la table "Enseignant"
CREATE TABLE Enseignant (
  id_enseignant INTEGER PRIMARY KEY,
  FOREIGN KEY (id_enseignant) REFERENCES Personne (id_personne)
);

-- Création de la table "Etudiant"
CREATE TABLE Etudiant (
  id_etudiant INTEGER PRIMARY KEY,
  FOREIGN KEY (id_etudiant) REFERENCES Personne (id_personne)
);
-- Création de la table "Module"
CREATE TABLE Module (
  id_module INTEGER PRIMARY KEY,
  intitule_module VARCHAR,
  code VARCHAR,
  ue VARCHAR,
  id_enseignant INTEGER,
  FOREIGN KEY (id_enseignant) REFERENCES Enseignant (id_enseignant)
);

-- Création de la table "Evaluation"
CREATE TABLE Evaluation (
  nom_evaluation VARCHAR,
  date_evaluation DATE,
  id_module INTEGER,
  coefficient FLOAT,
  echelle INTEGER,
  PRIMARY KEY (nom_evaluation, date_evaluation),
  FOREIGN KEY (id_module) REFERENCES Module (id_module)
);

-- Création de la table "Notation"
CREATE TABLE Notation (
  id_etudiant INTEGER,
  nom_evaluation VARCHAR,
  date_evaluation DATE,
  note FLOAT,
  FOREIGN KEY (id_etudiant) REFERENCES Etudiant (id_etudiant),
  FOREIGN KEY (nom_evaluation, date_evaluation) REFERENCES Evaluation
(nom_evaluation, date_evaluation),
  PRIMARY KEY (id_etudiant, nom_evaluation, date_evaluation)
);
```

```
           Liste des relations
 SchÚma  |     Nom      | Type  | PropriÚtaire
-------+-------------+-------+-------------
 public | enseignant  | table | postgres
 public | etudiant    | table | postgres
 public | evaluation  | table | postgres
 public | grandetable | table | postgres
 public | module      | table | postgres
 public | notation    | table | postgres
 public | personne    | table | postgres
(7 lignes)
```

Etape 4 : Une fois les tables créées, nous avons effectué la répartition des données présentes dans la table "grandetable" vers les tables correspondantes. Cela nous a permis de stocker les informations dans les tables spécifiques, en respectant les relations et les clés étrangères définies.

```sql
-- Insertion des données dans la table "Personne"
INSERT INTO Personne (id_personne, nom, prenom)
SELECT DISTINCT id_enseignant, nom_enseignant, prenom_enseignant
FROM grandetable
UNION
SELECT DISTINCT id_etudiant, nom_etudiant, prenom_etudiant
FROM grandetable;

-- Insertion des données dans la table "Enseignant"
INSERT INTO Enseignant (id_enseignant)
SELECT DISTINCT id_enseignant
FROM grandetable;

-- Insertion des données dans la table "Etudiant"
INSERT INTO Etudiant (id_etudiant)
SELECT DISTINCT id_etudiant
FROM grandetable;

-- Insertion des données dans la table "Module"
INSERT INTO Module (id_module, intitule_module, code, ue, id_enseignant)
SELECT DISTINCT id_module, intitule_module, code, ue, id_enseignant
FROM grandetable;

-- Insertion des données dans la table "Evaluation"
INSERT INTO Evaluation (nom_evaluation, date_evaluation, coefficient, echelle,
id_module)
SELECT DISTINCT nom_evaluation, date_evaluation, coefficient, echelle, id_module
FROM grandetable;

-- Insertion des données dans la table "Notation"
INSERT INTO Notation (id_etudiant, nom_evaluation, date_evaluation, note)
SELECT DISTINCT id_etudiant, nom_evaluation, date_evaluation, note
FROM grandetable;
```

```
INSERT 0 6087
```

Etape 5 : Enfin, afin de maintenir la structure et l'organisation des données, nous avons supprimé la table temporaire "grandetable" qui contenait l'ensemble des données initiales.

```sql
-- Suppression de la table "grandetable"
DROP TABLE IF EXISTS grandetable;
```

```
             Liste des relations
 SchÚma |     Nom      | Type  | PropriÚtaire
--------+--------------+-------+--------------
 public | enseignant   | table | postgres
 public | etudiant     | table | postgres
 public | evaluation   | table | postgres
 public | module       | table | postgres
 public | notation     | table | postgres
 public | personne     | table | postgres
(6 lignes)
```

Dans cette section, la création de la table "grandetable" a été réalisé par Sidar. Abou à effectuer l'insertion des données dans la table "grandetable". Le reste de la section, comprenant la création des tables, l'insertion des données dans chaque table et la suppression de la table "grandetable", a été réalisé en collaboration.

4. Requête particulièrement intéressante

Première requête (réalisé par Abou)

Explication, objectif et intérêt de la requête

La requête a pour objectif de calculer la moyenne normalisée des notes pour chaque évaluation, en prenant en compte à la fois le coefficient et l'échelle de chaque évaluation. Cette moyenne normalisée exprimée sur une échelle de 0 à 20 permet de comparer les performances des évaluations de manière équitable. Les résultats obtenus sont ensuite triés par ordre décroissant de la moyenne normalisée, ce qui permet d'identifier les évaluations les mieux notées. La requête limite également les résultats aux 10 premières lignes, ce qui offre un aperçu des évaluations les plus performantes. Ainsi, cette requête permet de classer les évaluations en fonction de leur moyenne normalisée, fournissant ainsi une mesure objective de la qualité des résultats obtenus.

Requête en question

```
SELECT Evaluation.nom_evaluation,
       Evaluation.date_evaluation,
       SUM(note * coefficient / echelle) / NULLIF(SUM(coefficient),
0) * 20 AS moyenne_normalisee_note,
       Module.intitule_module
FROM Evaluation
JOIN Notation USING(nom_evaluation,date_evaluation)
JOIN MODULE ON Evaluation.id_module = Module.id_module
WHERE coefficient <> 0
GROUP BY Evaluation.nom_evaluation,
         Evaluation.date_evaluation,
         Module.intitule_module
ORDER BY moyenne_normalisee_note DESC
LIMIT 10;
```

Copie d’écran avec le résultat de la requête

```
postgres=# SELECT Evaluation.nom_evaluation,
postgres-#        Evaluation.date_evaluation,
postgres-#        SUM(note * coefficient / echelle) / NULLIF(SUM(coefficient), 0) * 20 AS moyenne_normalisee_note,
postgres-#        Module.intitule_module
postgres-# FROM Evaluation
postgres-# JOIN Notation USING(nom_evaluation,date_evaluation)
postgres-# JOIN MODULE ON Evaluation.id_module = Module.id_module
postgres-# WHERE coefficient <> 0
postgres-# GROUP BY Evaluation.nom_evaluation,
postgres-#          Evaluation.date_evaluation,
postgres-#          Module.intitule_module
postgres-# ORDER BY moyenne_normalisee_note DESC
postgres-# LIMIT 10;
         nom_evaluation          | date_evaluation | moyenne_normalisee_note |               intitule_module
---------------------------------+-----------------+-------------------------+---------------------------------------------
 retour mat├®riel                | 2021-12-13      |      19.380165289256198 | Installation d'un poste pour le dveloppement
 compte-rendu                    | 2021-12-13      |      16.780172413793103 | Installation d'un poste pour le dveloppement
 Evaluation phase 1              | 2021-09-06      |      16.313333333333322 | Cration d'une base de donnes
 Mini-test 2 Relations           | 2021-10-04      |      16.276229508196714 | Mathmatiques discrtes
 Evaluation phase 3              | 2021-09-06      |      16.172173913043515 | Cration d'une base de donnes
 Conception d'un article (Draco) | 2022-01-11      |      15.524999999999999 | Bases de la communication
 Mini-test Pr├®dicats            | 2021-11-22      |      15.241477272727273 | Mathmatiques discrtes
 Participation                   | 2021-09-06      |      15.155555555555555 | Mathmatiques discrtes
 Projet                          | 2021-11-10      |      14.758870967741938 | Implmentation d'un besoin client
 Evaluation phase 2              | 2021-09-06      |      14.613333333333332 | Cration d'une base de donnes
(10 lignes)
```

Commentaire

Le résultat de la requête fournit un aperçu des performances des étudiants lors des évaluations. Les 10 premières évaluations affichées présentent des moyennes normalisées variables, allant de 19.38 à 14.61, ce qui indique une diversité dans les performances des étudiants. Certaines évaluations se démarquent avec des moyennes élevées, telles que "retour matériel" (19.38) et "compte-rendu" (16.78), suggérant de bonnes performances dans ces domaines spécifiques. Les modules associés aux évaluations varient, avec des domaines d'études tels que "Installation d'un poste pour le développement", "Création d'une base de données", "Mathématiques discrètes" et "Bases de la communication". La présence de différentes dates d'évaluation suggère une distribution sur une période donnée. En résumé, le résultat permet de comprendre les domaines où les étudiants se sont distingués et ceux où ils pourraient avoir besoin d'amélioration, offrant ainsi un aperçu global des performances des étudiants lors des évaluations.

Deuxième requête (réalisé par Sidar)

Explication, objectif et intérêt de la requête

La requête vise à calculer la moyenne normalisée des notes pour chaque module, en utilisant les notations correspondantes. L'objectif principal est de classer les modules en fonction de leur moyenne normalisée afin d'évaluer la performance des enseignants qui les enseignent. Cette requête est intéressante car elle prend en compte à la fois les notes attribuées aux évaluations et les coefficients associés à chaque évaluation. La normalisation des notes permet de pondérer les résultats en fonction de l'échelle utilisée, ce qui rend les comparaisons entre les modules plus équitables. Cela permet d'évaluer la performance des enseignants dans chaque module et d'identifier les modules où les étudiants

excellent ainsi ceux qui rencontrent des difficultés. Ces informations sont précieuses pour effectuer des ajustements pédagogiques, offrir un soutien supplémentaire aux enseignants et reconnaître et récompenser les enseignants performants. Elles permettent de mettre en place des mesures ciblées pour améliorer l'apprentissage des étudiants et favoriser leur réussite, tout en valorisant le travail des enseignants qui se démarquent.

Requête en question

```
SELECT Module.id_module,
       Module.intitule_module,
       CONCAT(Personne.nom, ' ', Personne.prenom) AS enseignant,
       SUM(note * coefficient / echelle) / SUM(coefficient) * 20 AS
moyenne_normalisee_note
FROM MODULE
JOIN Evaluation USING(id_module)
JOIN Enseignant USING(id_enseignant)
JOIN Notation USING(nom_evaluation,date_evaluation)
JOIN Personne ON Personne.id_personne = Enseignant.id_enseignant
GROUP BY Module.id_module,
         Module.intitule_module,
         enseignant
ORDER BY moyenne_normalisee_note;
```

Copie d'écran avec le résultat de la requête

```
postgres=# SELECT Module.id_module,
postgres-#        Module.intitule_module,
postgres-#        CONCAT(Personne.nom, ' ', Personne.prenom) AS enseignant,
postgres-#        SUM(note * coefficient / echelle) / SUM(coefficient) * 20 AS moyenne_normalisee_note
postgres-# FROM Module
postgres-# JOIN Evaluation using(id_module)
postgres-# JOIN Enseignant using(id_enseignant)
postgres-# JOIN Notation using(nom_evaluation,date_evaluation)
postgres-# JOIN Personne ON Personne.id_personne = Enseignant.id_enseignant
postgres-# GROUP BY Module.id_module, Module.intitule_module, enseignant
postgres-# ORDER BY moyenne_normalisee_note;
 id_module |                      intitule_module                      |      enseignant      | moyenne_normalisee_note
-----------+-----------------------------------------------------------+----------------------+-------------------------
        18 | Outils mathmatiques fondamentaux                          | Gillet Cyril         |       9.533667621776505
         8 | Introduction  l'architecture des ordinateurs              | Becourt Nicole       |      10.020437685459946
        13 | Initiation au dveloppement                                | Martz Patrick        |      10.833616767685587
         7 | Introduction aux bases de donnes et SQL                   | Larramendy Francisca |      10.941422594142256
        16 | Comparaison d'approches algorithmiques                    | Chaverot Krimo       |      11.011825552138658
        10 | Anglais technique                                         | Loncke Albert        |      11.133290267011194
         6 | Introduction aux systmes dexploitation et  leur fonctionnement | Nicolai Radoine      |       11.25308641975309
         2 | Dveloppement d'interfaces web                             | Dantan Renee         |      11.287569721115538
        15 | conomie durable et numrique                               | Barret Wiliam        |      11.611445783132524
        12 | Implmentation d'un besoin client                          | Gac Francois         |      11.678644501278772
         1 | Recueil de besoins                                        | Di Placido Arnaud    |       11.91851851851852
         5 | Mathmatiques discrtes                                     | Renard Romain        |      12.998017171917237
         9 | Bases de la communication                                 | Mouette Henri        |      13.027173913043486
        11 | Gestion de projet & des organisations                     | Vasseur Florence     |      13.171548672566365
         3 | Dcouverte de l'environnement conomique et cologique       | Preira Chrystel      |       13.66525423728814
        14 | Projet professionnel et personnel                         | Creantor Andre       |      13.665919282511208
         4 | Cration d'une base de donnes                              | Crapez Annie         |      14.083333333333288
        17 | Installation d'un poste pour le dveloppement              | Mailhe Maurice       |      15.260971055088703
(18 lignes)
```

Commentaire

Les résultats présentent les moyennes normalisées des notes pour chaque module, triées par ordre croissant. La colonne

"moyenne_normalisee_note" indique la note moyenne pondérée du module sur une échelle de 20. On peut observer que le module "Outils mathématiques fondamentaux", enseigné par Gillet Cyril, obtient la moyenne des notes la plus basse avec 9.53, tandis que le module "Installation d'un poste pour le développement", enseigné par Mailhe Maurice, obtient la moyenne la plus élevée avec 15.26. Ces résultats permettent d'identifier les enseignants qui enseignent efficacement et les étudiants qui obtiennent de bons résultats dans chaque module, ainsi que ceux qui pourraient avoir besoin d'un soutien supplémentaire.